GEORECKON

Intelligent Dead Reckoning Engine — SIH26168

Kotlin Android App ↔ Python Backend

Complete Integration Guide — All Methods, Pros & Cons, Code Examples

Document Type	Technical Integration Guide
Project	GeoReckon v3.2 Production Engine
Hackathon	ISRO SIH 2026 — Problem SIH26168

RECOMMENDED METHOD AT A GLANCE

- Run GeoReckon as a Flask/SocketIO WebSocket server on your laptop or cloud
- Kotlin app reads phone sensors (Accel, Gyro, Gravity) at 10 Hz
- Kotlin sends sensor JSON to server via OkHttp WebSocket every 100ms
- Server processes data through all 6 pipeline stages and returns position
- Kotlin receives { lat, lon, speed, confidence } and updates the app map

1. Overview — The Integration Problem

GeoReckon is written in Python and runs on a laptop or cloud server. Your Android navigation app is written in Kotlin and runs on a smartphone. These two need to communicate so the phone's IMU sensors can feed the GeoReckon pipeline and the computed position can update the map in real time.

The Data Flow

Step	What Happens	Who Does It
1	Phone reads Accelerometer + Gyroscope + Gravity at 10 Hz	Kotlin App
2	GPS is lost (tunnel/underpass) — dead reckoning begins	Kotlin detects
3	Kotlin sends {accel, gyro, gravity, dt} as JSON every 100ms	Kotlin → Server
4	Server runs 6-stage GeoReckon pipeline on received data	Python Server
5	Server returns {position_enu, speed_kmh, confidence} instantly	Server → Kotlin
6	Kotlin updates map dot with estimated position	Kotlin App

Methods Comparison at a Glance

Method	Difficulty	Offline?	No Port Needed?	Best For
■ WebSocket (Recommended)	■ Easy	No	Yes	Hackathon demo, real-time dashboard
REST API (HTTP)	■ Easy	No	Yes	Simple polling, non-real-time
Chaquopy (Python on Android)	■■ Medium	Yes	Yes	Offline prototype — but large APK
Native Kotlin Port	■■■ Hard	Yes	No	Production app — requires full rewrite
JNI / C++ NDK	■■■■ Very Hard	Yes	No	High-performance — not feasible for SIH

2. Recommended Method — WebSocket Bridge ■

WebSocket gives a persistent two-way connection between the Kotlin app and the Python server. Unlike HTTP (which opens a new connection for every request), WebSocket stays open continuously — perfect for streaming 10 sensor readings per second with minimal overhead.

Why WebSocket over plain HTTP?

- HTTP adds 30–100ms overhead per request (TCP handshake + headers) → too slow for 10 Hz streaming
- WebSocket adds only 2–5ms per message after the initial connection — fast enough for real-time DR
- WebSocket is bidirectional — server can push position updates without the app polling
- Flask-SocketIO library makes this trivial to set up in Python (4 lines of code)
- OkHttp WebSocket client in Kotlin is stable, production-grade, already in most Android apps

Pros & Cons

■ PROS	■ CONS
• Real-time — sub 10ms latency per message	• Requires internet/WiFi (no offline in tunnel)
• Bidirectional — server pushes to app	• Python server must be running
• No phone restart needed — stays connected	• If WiFi drops, must reconnect
• Works over WiFi and mobile data (4G/5G)	• For production: needs a hosted cloud server
• Works with ngrok for instant public URL	
• No Android code changes needed for pipeline updates	
• Can connect multiple phones simultaneously	

Step-by-Step Setup

Step 1 — Install Python dependencies

```
// Terminal — run in your project folder
pip install flask flask-socketio flask-cors
```

Step 2 — Create server.py in your GeoReckon project

```
// georeckon/web_server.py — complete server file
# georeckon/web_server.py
from flask import Flask
from flask_socketio import SocketIO, emit
from flask_cors import CORS
import numpy as np, sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
from georeckon.src.virtual_alignment   import VirtualInVehicleAligner
from georeckon.src.vibration_filter    import KinematicVibrationFilter
from georeckon.src.kinematics_speed_estimator import KinematicSpeedEstimator
app = Flask(__name__)
CORS(app)
socketio = SocketIO(app, cors_allowed_origins='*', async_mode='threading')
aligner    = VirtualInVehicleAligner()
vib_filter = KinematicVibrationFilter(sampling_rate=10.0)
```

```
speed_est  = KinematicSpeedEstimator(dt=0.1)
state      = {'pos': [0.0, 0.0], 'heading': 0.0, 'mode': 'GPS'}
accel_buf, gyro_buf = [], []

@socketio.on('calibrate')
def on_calibrate(data):
    R, health = aligner.compute_alignment_from_history(
        np.array(data['gravity_h']), np.array(data['accel_h']),
        np.array(data['speed_h']),  np.array(data['time_h']))
    speed_est.sync_gnss_speed(float(data['entry_speed']))
    emit('calibrate_done', {'health': float(health)})

@socketio.on('sensor')
def on_sensor(data):
    global accel_buf, gyro_buf
    acc = np.array(data['accel'])
    gyr = np.array(data['gyro'])
    grav = np.array(data.get('gravity', [0,0,9.8]))
    dt  = float(data.get('dt', 0.1))
    accel_buf.append(acc); gyro_buf.append(gyr)
    if len(accel_buf) > 10: accel_buf.pop(0); gyro_buf.pop(0)
    ca, cw, shock = vib_filter.update_sample(acc, gyr)
    lv = aligner.R_b_to_v @ (ca - grav)
    wv = aligner.R_b_to_v @ cw
    spd, stopped = speed_est.estimate_step(
        float(lv[0]), float(lv[1]), float(wv[2]),
        np.array(accel_buf), np.array(gyro_buf), dt)
    state['heading']  += float(wv[2]) * dt
    state['pos'][0]   += spd * np.cos(state['heading']) * dt
    state['pos'][1]   += spd * np.sin(state['heading']) * dt
    emit('position', {
        'pos_enu':   state['pos'],
        'speed_kmh': round(spd * 3.6, 1),
        'heading':   float(np.degrees(state['heading'])),
        'mode':      'DR'
    })
if __name__ == '__main__':
    print('GeoReckon server at http://0.0.0.0:5000')
    socketio.run(app, host='0.0.0.0', port=5000)
```

Step 3 — Run the server

// Terminal

```
cd c:\Users\...\sih_Bhanu
python -m georeckon.web_server

# You will see:
# GeoReckon server at http://0.0.0.0:5000
# Your IP address (run: ipconfig) e.g. 192.168.1.42
```

3. Kotlin Android App — Connection Code

Step 4 — Add OkHttp dependency to build.gradle

```
// app/build.gradle
dependencies {
    // OkHttp for WebSocket
    implementation 'com.squareup.okhttp3:okhttp:4.12.0'
    // Gson for JSON serialization
    implementation 'com.google.code.gson:gson:2.10.1'
}
```

Step 5 — Add Internet permission to AndroidManifest.xml

```
// AndroidManifest.xml
<uses-permission android:name='android.permission.INTERNET'/>
<uses-permission android:name='android.permission.ACCESS_NETWORK_STATE'/>
```

Step 6 — Create GeoReckonClient.kt

```
// GeoReckonClient.kt
// GeoReckonClient.kt
import okhttp3.*
import org.json.JSONObject
import android.util.Log
class GeoReckonClient(private val serverUrl: String) {
    // serverUrl example: 'ws://192.168.1.42:5000/socket.io/?EIO=4&transport;=websocket'
    private var ws: WebSocket? = null
    private val client = OkHttpClient()
    var onPosition: ((Double, Double, Double, Double) -> Unit)? = null
    // callback: (easting_m, northing_m, speed_kmh, heading_deg)
    fun connect() {
        val request = Request.Builder().url(serverUrl).build()
        ws = client.newWebSocket(request, object : WebSocketListener() {
            override fun onOpen(webSocket: WebSocket, response: Response) {
                Log.d('GeoReckon', 'Connected to server')
                // Send connect event (Socket.IO protocol)
                webSocket.send('40')  // Socket.IO CONNECT packet
            }
            override fun onMessage(webSocket: WebSocket, text: String) {
                handleMessage(text)
            }
            override fun onFailure(webSocket: WebSocket, t: Throwable, r: Response?) {
                Log.e('GeoReckon', 'Connection failed: ${t.message}')
            }
        })
    }
    fun sendSensorData(accel: FloatArray, gyro: FloatArray,
                       gravity: FloatArray, dt: Float) {
        val json = JSONObject().apply {
            put('accel',   accel.toList())
            put('gyro',    gyro.toList())
            put('gravity', gravity.toList())
```

```
            put('dt',      dt)
        }
        // Socket.IO event format: 42["sensor", {data}]
        ws?.send('42["sensor",$json]')
    }
    fun sendCalibration(gravityH: List<List<Float>>, accelH: List<List<Float>>,
                        speedH: List<Float>, timeH: List<Float>,
                        entrySpeed: Float) {
        val json = JSONObject().apply {
            put('gravity_h',   gravityH)
            put('accel_h',     accelH)
            put('speed_h',     speedH)
            put('time_h',      timeH)
            put('entry_speed', entrySpeed)
        }
        ws?.send('42["calibrate",$json]')
    }
    private fun handleMessage(text: String) {
        // Filter Socket.IO position events: 42["position", {...}]
        if (text.startsWith('42["position"')) {
            val payload = text.substringAfter(',').dropLast(1)
            val data    = JSONObject(payload)
            val posEnu  = data.getJSONArray('pos_enu')
            onPosition?.invoke(
                posEnu.getDouble(0),           // east meters
                posEnu.getDouble(1),           // north meters
                data.getDouble('speed_kmh'),   // km/h
                data.getDouble('heading')      // degrees
            )
        }
    }
    fun disconnect() { ws?.close(1000, 'User closed'); client.dispatcher.executorService.shutdown()
}
}
```

4. Using It in Your MainActivity.kt

Step 7 — Wire sensors + GeoReckon client in your Activity

// MainActivity.kt

```
// MainActivity.kt (relevant parts)
class MainActivity : AppCompatActivity(), SensorEventListener {

    // ■■■ 1. Setup ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    private val SERVER_URL = 'ws://192.168.1.42:5000/socket.io/?EIO=4&transport;=websocket'
    // ↑ Replace with your laptop IP after running: ipconfig

    private lateinit var geoReckon: GeoReckonClient
    private lateinit var sensorManager: SensorManager
    private var isDRMode = false
    private var lastAccel   = floatArrayOf(0f, 0f, 9.8f)
    private var lastGyro    = floatArrayOf(0f, 0f, 0f)
    private var lastGravity = floatArrayOf(0f, 0f, 9.8f)
    private var lastTime    = 0L

    // History buffers for calibration (last 60 seconds)
    private val accelHistory   = ArrayDeque<FloatArray>()
    private val gravityHistory = ArrayDeque<FloatArray>()
    private val speedHistory   = ArrayDeque<Float>()
    private val timeHistory    = ArrayDeque<Float>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // ■■■ Connect to GeoReckon server ■■■■■■■■■■■■■
        geoReckon = GeoReckonClient(SERVER_URL)
        geoReckon.onPosition = { east, north, speed, heading ->
            runOnUiThread {
                updateMapMarker(east, north)   // your map update function
                updateSpeedDisplay(speed)
            }
        }
        geoReckon.connect()

        // ■■■ Register sensors at 10 Hz (100,000 µs) ■
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        sensorManager.registerListener(this,
            sensorManager.getDefaultSensor(Sensor.TYPE_LINEAR_ACCELERATION), 100_000)
        sensorManager.registerListener(this,
            sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE), 100_000)
        sensorManager.registerListener(this,
            sensorManager.getDefaultSensor(Sensor.TYPE_GRAVITY), 100_000)
    }

    override fun onSensorChanged(event: SensorEvent) {
        when (event.sensor.type) {
            Sensor.TYPE_LINEAR_ACCELERATION -> lastAccel   = event.values.clone()
            Sensor.TYPE_GYROSCOPE           -> lastGyro    = event.values.clone()
            Sensor.TYPE_GRAVITY             -> lastGravity = event.values.clone()
        }
        // Store history for calibration (max 600 samples = 60s at 10Hz)
        if (!isDRMode) {
            accelHistory.addLast(lastAccel.clone())
```

```
            gravityHistory.addLast(lastGravity.clone())
            if (accelHistory.size > 600) { accelHistory.removeFirst(); gravityHistory.removeFirst()
}
        }
        // Send to GeoReckon every 100ms when in DR mode
        val now = System.currentTimeMillis()
        if (isDRMode && now - lastTime >= 100) {
            lastTime = now
            geoReckon.sendSensorData(lastAccel, lastGyro, lastGravity, 0.1f)
        }
    }
    // ■■■ Call this when GPS signal is lost ■■■■■■■■■■
    fun onGpsLost(lastKnownSpeedMs: Float) {
        isDRMode = true
        // Send calibration data to server
        geoReckon.sendCalibration(
            gravityHistory = gravityHistory.map { it.toList() },
            accelH  = accelHistory.map { it.toList() },
            speedH  = speedHistory.toList(),
            timeH   = timeHistory.toList(),
            entrySpeed = lastKnownSpeedMs
        )
    }
    // ■■■ Call this when GPS comes back ■■■■■■■■■■■■■■■■
    fun onGpsRestored() {
        isDRMode = false
    }
    override fun onDestroy() {
        super.onDestroy()
        geoReckon.disconnect()
        sensorManager.unregisterListener(this)
    }
    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
}
```

5. Hosting — Making It Publicly Accessible

Option A — Demo Day (ngrok) — Easiest ■

ngrok creates a public HTTPS/WSS tunnel to your laptop in one command. Perfect for demos where you need a public URL instantly. Free tier works fine.

// Terminal — run after starting your server

```
# Install ngrok: https://ngrok.com/download (free account)
ngrok http 5000
# ngrok will print something like:
# Forwarding  https://abc123.ngrok-free.app -> http://localhost:5000
# Forwarding  wss://abc123.ngrok-free.app   -> ws://localhost:5000
# In your Kotlin app, change SERVER_URL to:
# wss://abc123.ngrok-free.app/socket.io/?EIO=4&transport;=websocket
# Now your phone can connect from ANYWHERE — demo on 4G, not just WiFi!
```

Option B — Same WiFi (Direct IP) — Fastest

// Terminal

```
# Find your laptop IP:
ipconfig         # Windows
ifconfig         # Mac/Linux
# Look for IPv4 Address, e.g. 192.168.1.42
# In Kotlin: SERVER_URL = 'ws://192.168.1.42:5000/socket.io/?...'
# Phone and laptop must be on same WiFi
```

Option C — Cloud Server (Production) — Most Robust

For a production app or if you want the server always available without a laptop:

- Railway.app (free tier) — Deploy with 'railway up' in one command
- Render.com (free tier) — Connect GitHub repo, auto-deploy
- Google Cloud Run — Scales to zero when not in use, free quota
- DigitalOcean Droplet — $6/month, full control

// Railway deployment (cloud hosting in 3 commands)

```
npm install -g @railway/cli
railway login
railway up
# Railway gives you a permanent URL like: https://georeckon.up.railway.app
# Update SERVER_URL in Kotlin to this URL (wss://georeckon.up.railway.app/socket.io/...)
```

6. Other Methods — Why They Are Harder / Less Feasible

Method 2 — REST API (HTTP Polling)

Instead of a persistent WebSocket, the Kotlin app makes an HTTP POST request every 100ms. This works but is significantly slower and more wasteful.

Why it is LESS FEASIBLE for real-time DR:

- Each HTTP request opens a new TCP connection → 30–80ms overhead every call
- At 10 Hz you'd need 10 connections/second → 300–800ms latency overhead = useless
- Headers sent every request waste bandwidth (500+ bytes vs 50 bytes for WebSocket frame)
- Server cannot push to app — app must constantly poll even when nothing changed
- HTTP 1.1 keep-alive partially helps, but still far slower than WebSocket

→ VERDICT: Use HTTP only if you drop to 1 Hz or 2 Hz sampling (not suitable for DR accuracy)

■ PROS	■ CONS
• Simpler to implement — basic Retrofit calls	• 30–80ms overhead per request (too slow for 10 Hz)
• No special library needed	• Cannot push from server to app
• Works anywhere HTTP works	• Much higher battery drain from repeated connections
	• Not suitable for real-time dead reckoning

Method 3 — Chaquopy (Run Python On the Phone)

Chaquopy is a Gradle plugin that embeds a Python interpreter inside the Android APK. Your Python files literally run on the phone — no server needed.

Why it is LESS FEASIBLE for GeoReckon:

- APK size explodes to 80–150 MB (numpy alone is 40 MB) — users won't install this
- PyTorch (used by KinoNet) is NOT supported by Chaquopy at all
- scipy is not officially supported — the Butterworth filter won't work
- Python runs 5–10x slower on Android compared to laptop → may miss 100ms deadline
- Cannot update the pipeline without updating the entire APK through Play Store
- Debugging is extremely difficult (no proper Android debugger for Python)

→ VERDICT: Only feasible if you strip out KinoNet and scipy and only use numpy

■ PROS	■ CONS
• 100% offline — no server, no internet	• APK size 80–150 MB (too large for most users)
• No code changes to Python pipeline	• PyTorch not supported — KinoNet breaks
• Works even in tunnel with no connectivity	• scipy not officially supported — filter breaks
	• 5–10x slower execution on phone
	• Very hard to debug
	• Cannot update pipeline without app update

Method 4 — Native Kotlin Port (Rewrite Everything)

Port every Python file (geometry.py, virtual_alignment.py, filter_inekf.py, etc.) to native Kotlin/Java. The algorithm runs entirely on-device with no server.

Why it is LESS FEASIBLE for SIH hackathon timeframe:

- 6 Python source files with complex math must be completely rewritten in Kotlin
- The InEKF filter requires 15x15 matrix operations — no standard Kotlin matrix library
- scipy.signal.filtfilt (zero-phase filter) must be implemented from scratch
- numpy's eigh() (eigenvalue decomposition for PCA) must be manually implemented
- Total estimated effort: 6–10 weeks for a single developer (vs 1 day for WebSocket)
- Any bug in the Python pipeline must be fixed in TWO places (Python + Kotlin)
- SIH judges cannot easily verify the algorithm — the Python version is already audited

→ VERDICT: This is the right long-term production approach but NOT for a hackathon demo

■ PROS	■ CONS
• 100% offline — works anywhere including tunnels	• 6–10 weeks development time
• Best performance — native JVM code	• Full rewrite of all 6 pipeline modules
• No server dependency in production	• No scipy/numpy — all math from scratch
• Small APK size	• Bugs must be fixed in two codebases
	• InEKF 15x15 matrix math is very hard in Kotlin
	• NOT feasible within hackathon timeline

Method 5 — JNI / C++ NDK (Compile for Android)

Rewrite the core GeoReckon math in C++, compile it to a shared library (.so), and call it from Kotlin via the Android NDK's Java Native Interface (JNI).

Why it is EXTREMELY HARD and not feasible:

- Requires expert-level knowledge of C++, CMake, Android NDK, and JNI bindings
- Eigen (C++ matrix library) must be integrated — complex build configuration
- Memory management is manual (no garbage collection) → risk of crashes/leaks
- Debugging native crashes on Android requires specialized tools (LLDB, AddressSanitizer)
- Estimated effort: 12–16 weeks for an experienced C++ Android developer
- Cross-compilation toolchain setup alone can take days to get right

→ VERDICT: Used by production navigation apps (Google Maps, HERE) after years of development

→ VERDICT: Absolutely not feasible for a 36–48 hour hackathon

■ PROS	■ CONS
• Absolute maximum performance	• 12–16 weeks development time minimum
• Works offline	• Requires expert C++ + NDK + JNI knowledge
• Used by production-grade navigation apps	• Manual memory management — risk of crashes
	• Extremely complex debugging
	• Not remotely feasible for hackathon
	• Much harder than WebSocket for same result

7. Quick Reference — 30 Minute Setup

Everything you need to go from zero to connected in 30 minutes:

#	Action	Command / File
1	Install deps	`pip install flask flask-socketio flask-cors`
2	Add server	`Create georeckon/web_server.py (see Page 3)`
3	Run server	`python -m georeckon.web_server`
4	Find IP	`Run: ipconfig → note IPv4 e.g. 192.168.1.42`
5	Kotlin dep	`Add OkHttp: implementation 'com.squareup.okhttp3:okhttp:4.12.0'`
6	Manifest	`Add:`
7	Client code	`Add GeoReckonClient.kt to your project (see Page 4)`
8	Wire sensors	`Add sensor registration + geoReckon.sendSensorData() (see Page 5)`
9	Set URL	`SERVER_URL = 'ws://192.168.1.42:5000/socket.io/?EIO=4&transport;=websocket'`
10	Test!	`Run app on phone (same WiFi as laptop) → GPS loss → position updates!`

Troubleshooting

Problem	Fix
App can't connect to server	Check: same WiFi? Correct IP? Firewall blocking port 5000? Try: curl http://YOUR_IP:5000/health
No position updates received	Check: isDRMode = true before sending? Calibrate event sent first? Check Python server terminal logs
flask_socketio not found	Run: pip install flask-socketio flask-cors eventlet
CORS error in browser/app	Make sure CORS(app) is called and cors_allowed_origins='*' is set in SocketIO()
Want public URL without same WiFi	Use ngrok: install ngrok, run 'ngrok http 5000', use the wss:// URL in Kotlin